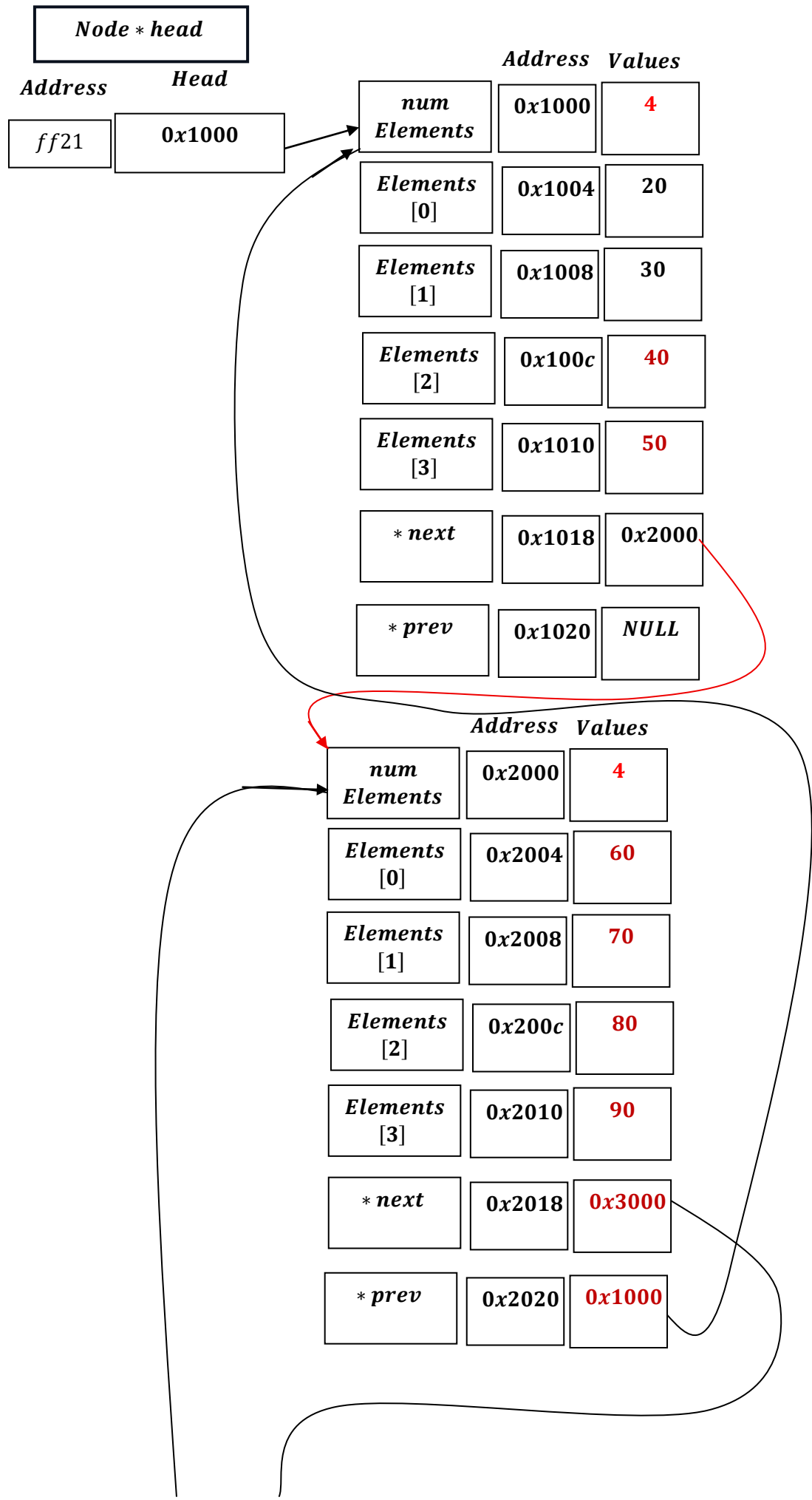


Unroll List – Doubly, Circular, Doubly – Circular

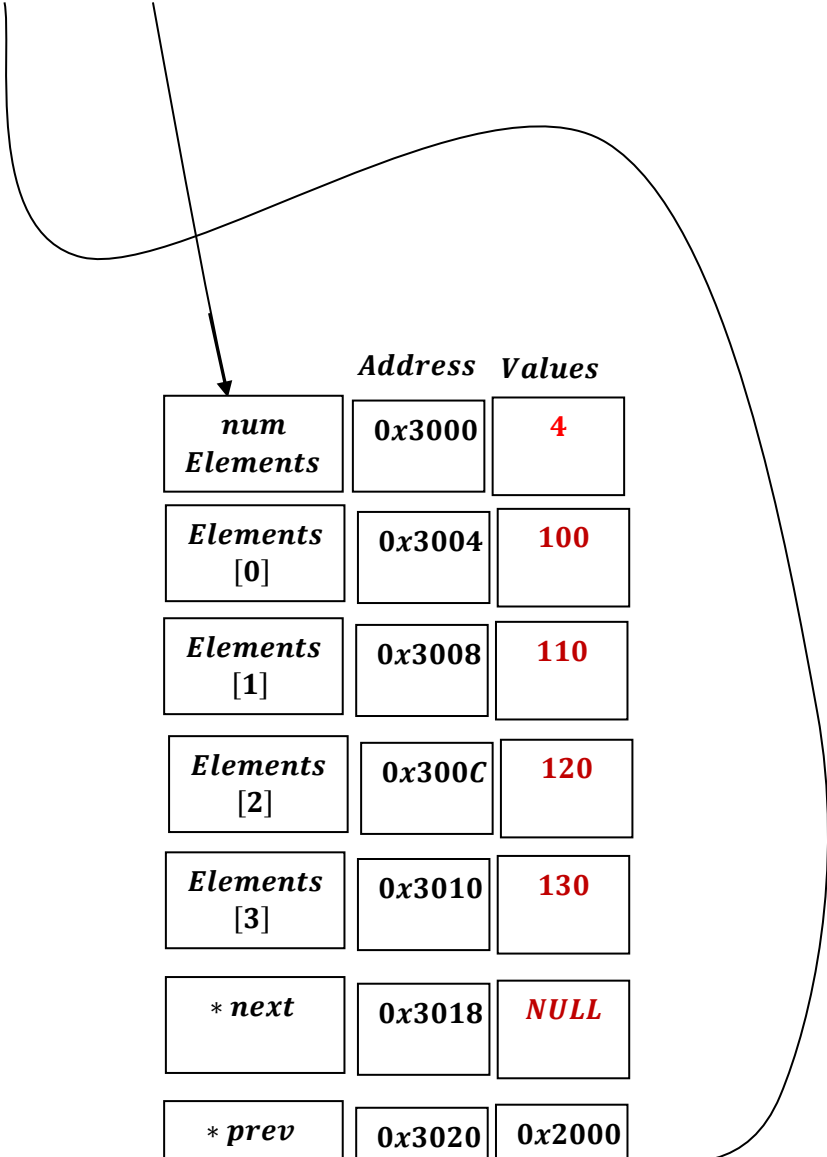
We have learnt about Singly Linked Unroll list ,now how would Doubly Linked Unroll List look like: –

1. Doubly Unrolled Linked List

```
#define MAX_ELEMENTS 4
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
    struct Node* prev; // New pointer for backward traversal
};
```

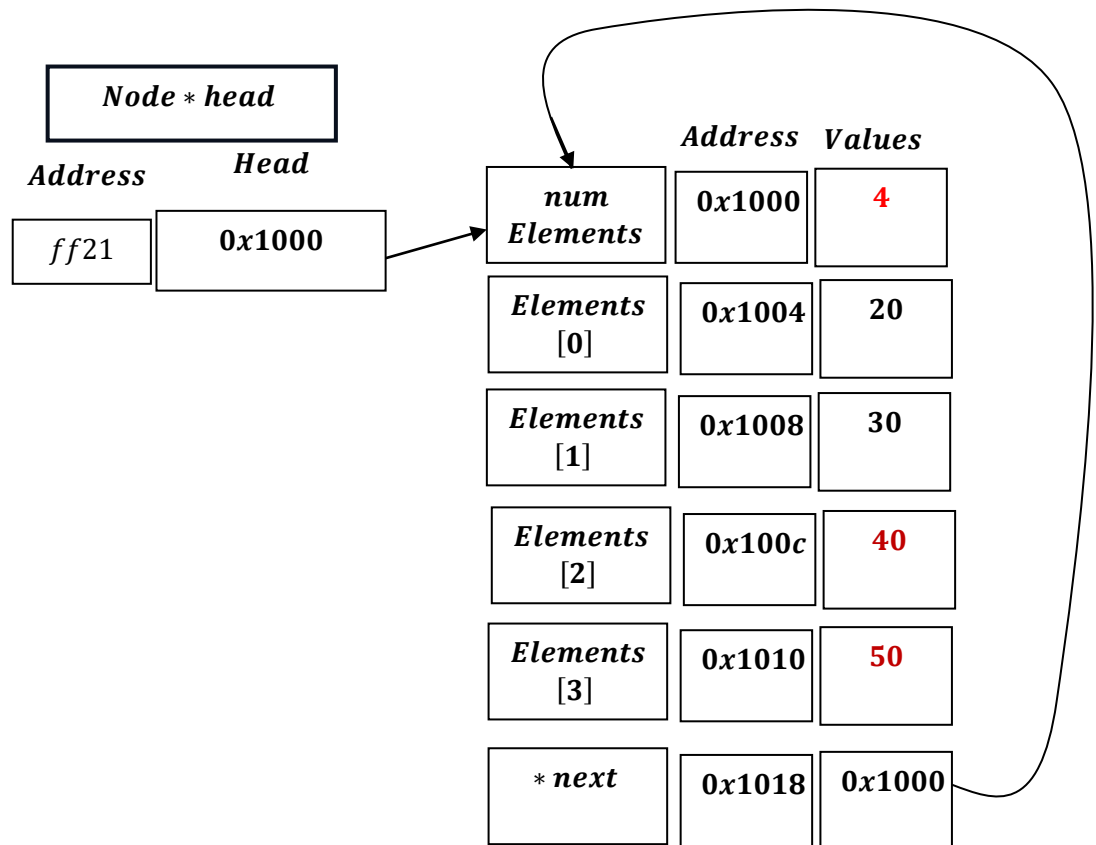


	<i>Address</i>	<i>Values</i>
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	NULL
* <i>prev</i>	0x3020	0x2000

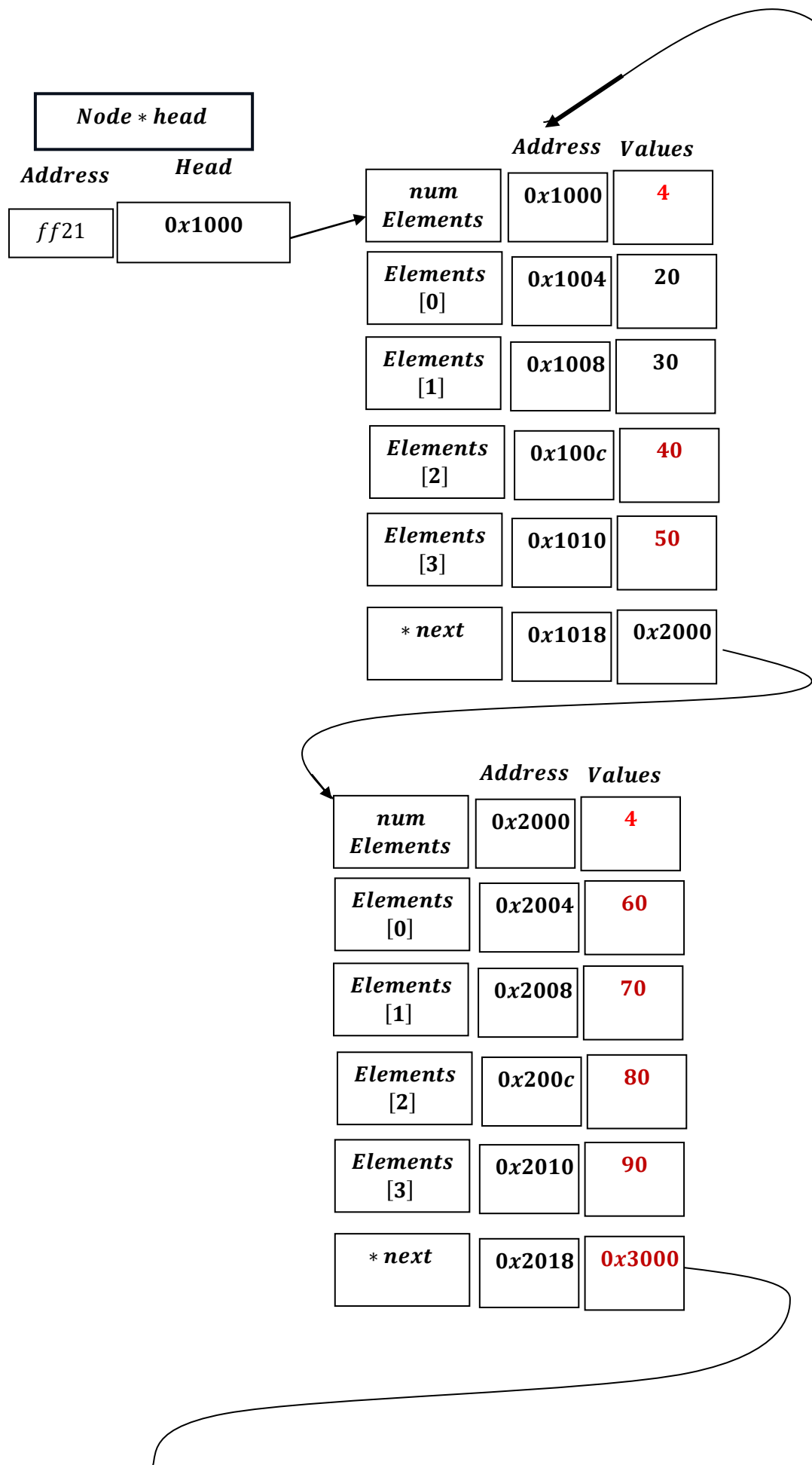


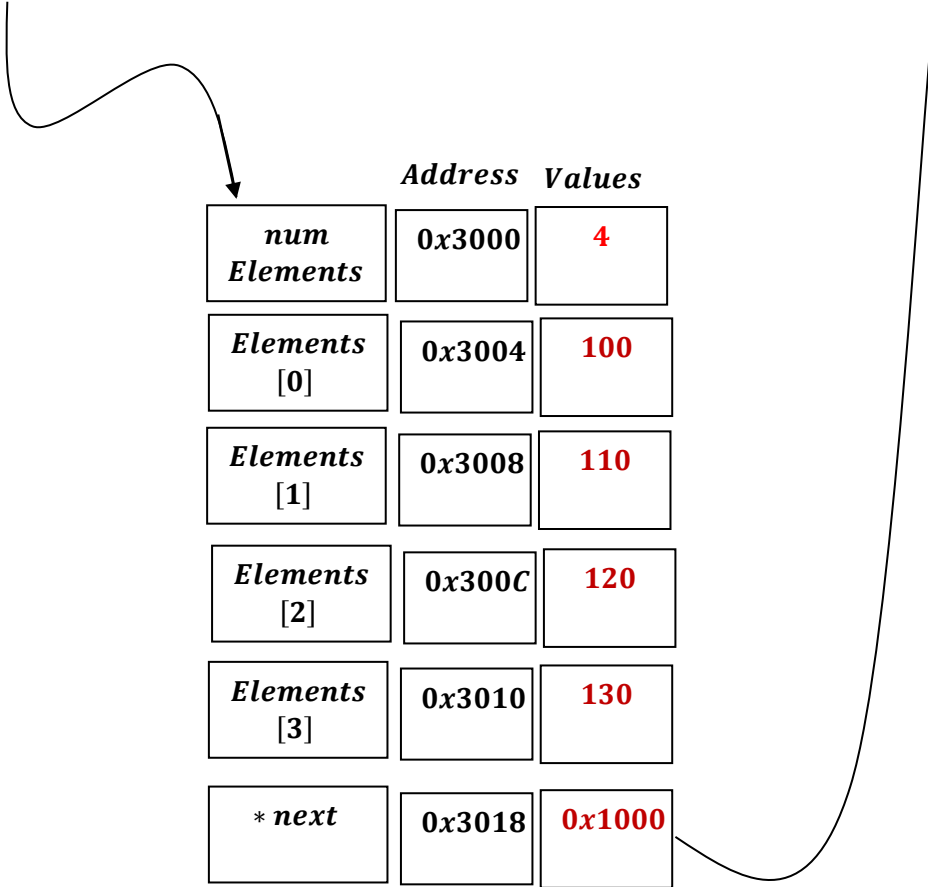
2. Circular Unrolled Linked List

2.A. With Single node:



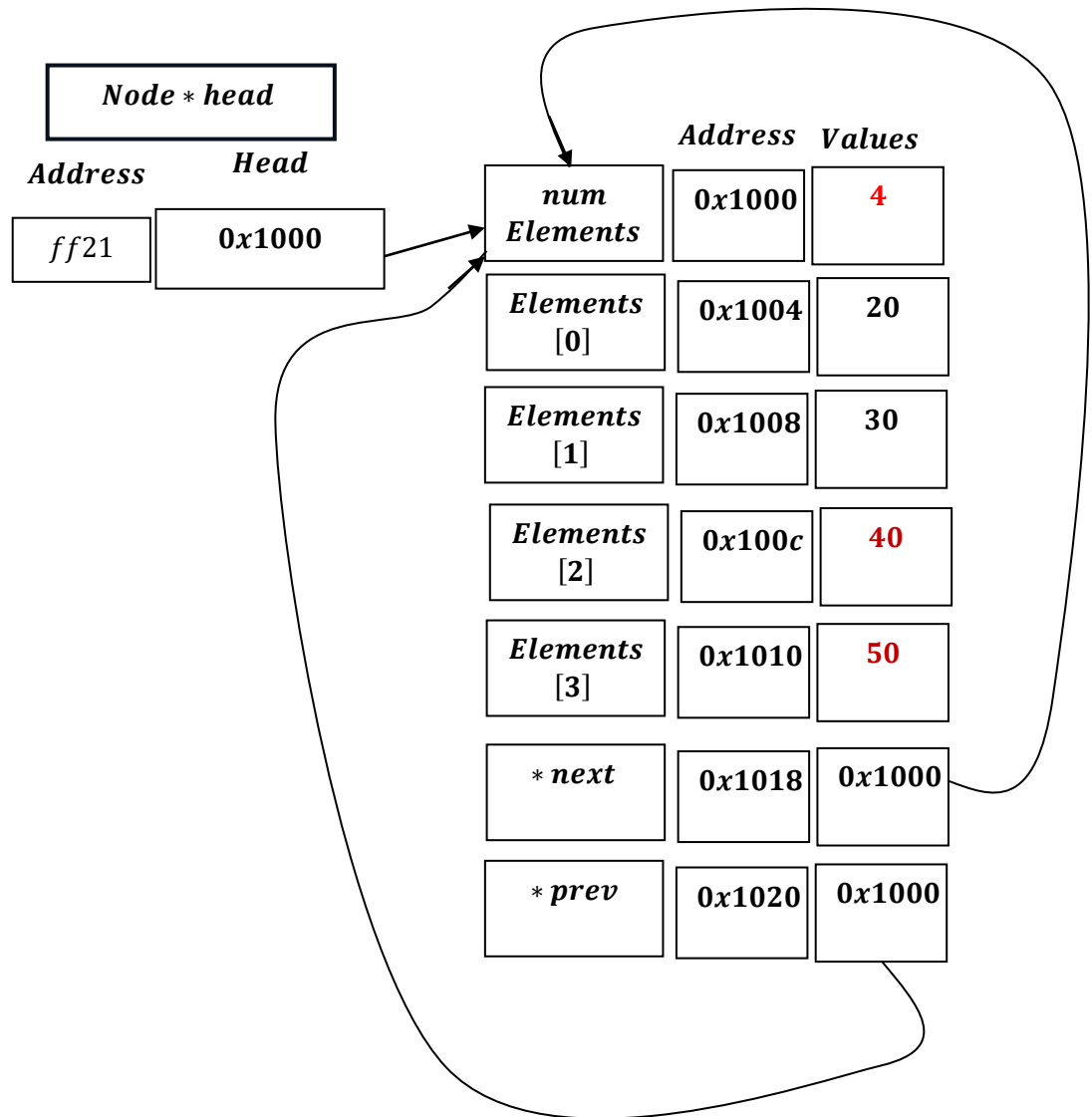
2. B. With Multiple nodes:



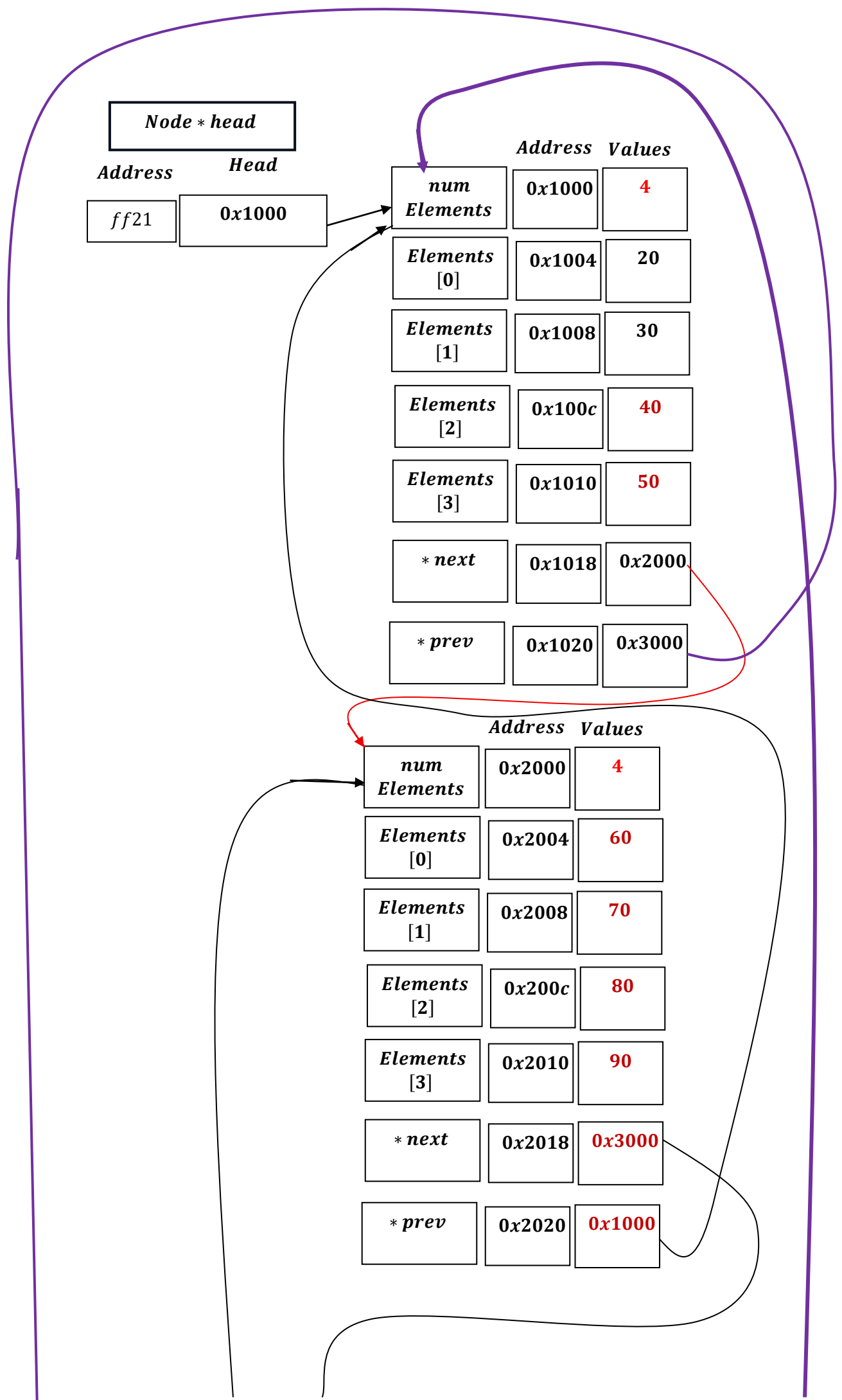


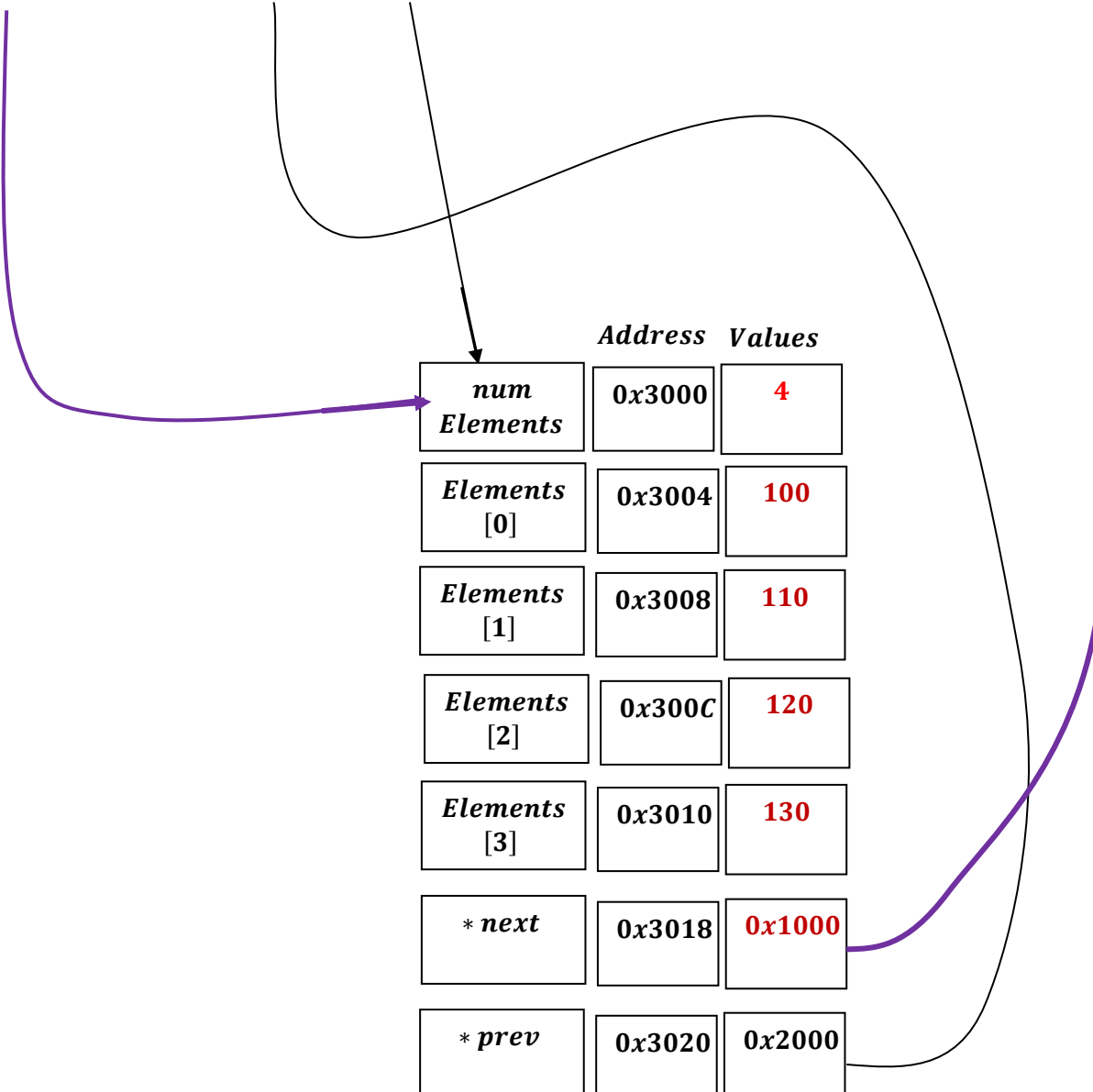
3. Circular – Doubly Unrolled Linked List

3.A. With Single node:



3.B. With Multiple nodes:





The diagram illustrates a linked list structure. A vertical table contains fields for the number of elements, individual elements, the next pointer, and the previous pointer. A purple arrow points from the top left to the 'num Elements' field. A black arrow points from the top center to the 'num Elements' field. A purple arrow points from the '* next' field to the right. A black arrow points from the '* prev' field to the right. A horizontal line is at the bottom of the page.

	<i>Address</i>	<i>Values</i>
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	0x1000
* <i>prev</i>	0x3020	0x2000